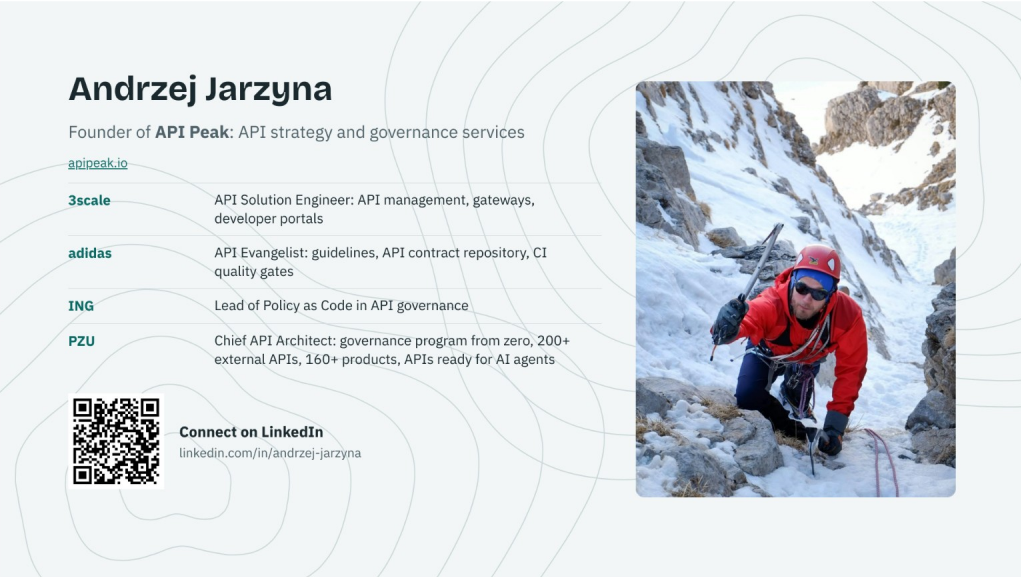


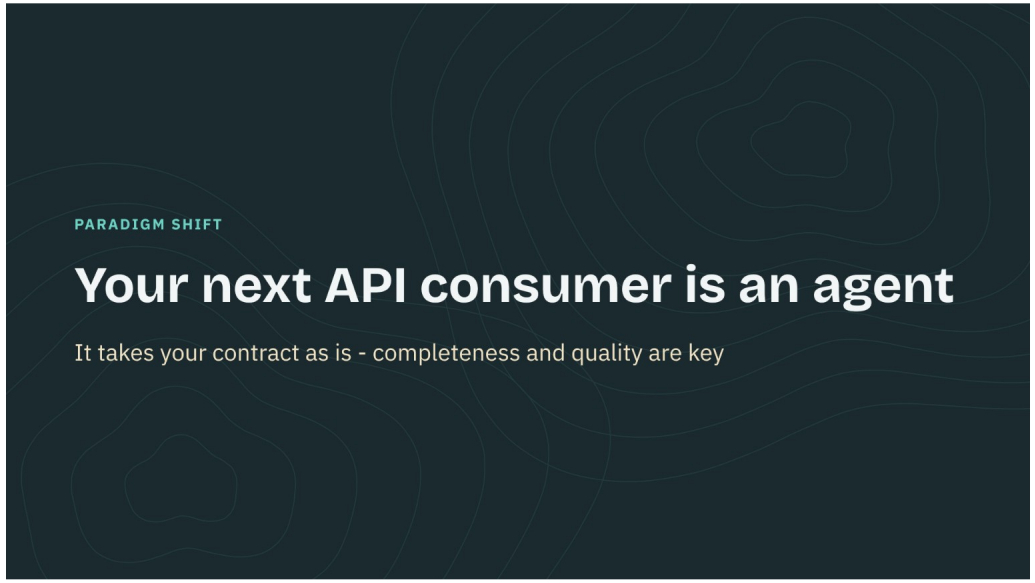
0:00 → 0:15. Good afternoon. I'm Andrzej, and for the next twenty minutes or so we'll build API governance into a delivery pipeline, one gate at a time, in a real, working repo. Transition: first, who I am, in thirty seconds.



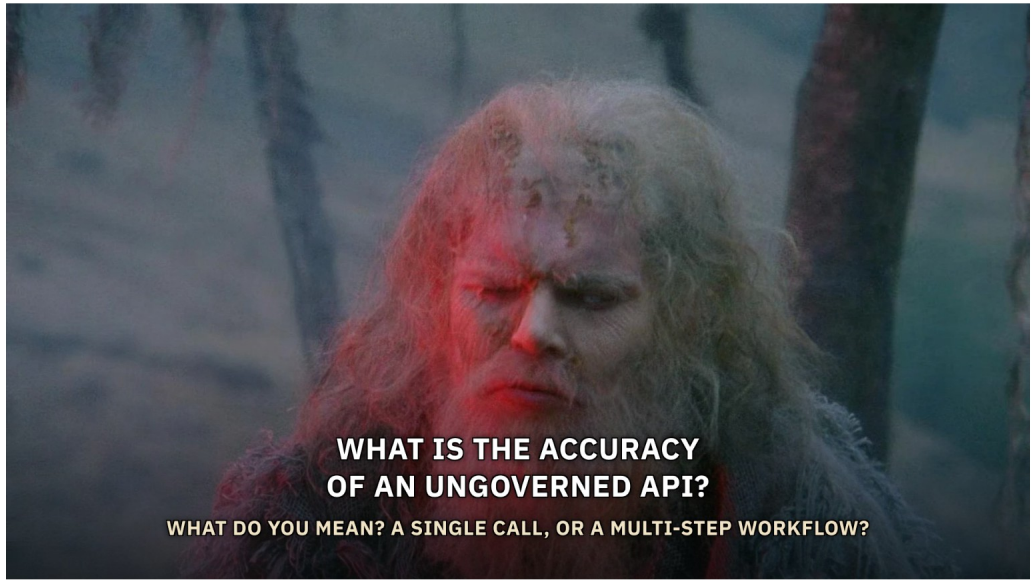
0:15 → 0:35. Founder of API Peak, my own company: API strategy and governance for companies; it's at apipeak.io. Before that I built API governance at adidas, ING and most recently PZU, the largest insurer in Central-Eastern Europe. Co-author of the RESTful API Design Patterns book. Fun fact: API Peak is also the made-up company in today's demo repo. The QR code is my LinkedIn, happy to connect. Transition: and one line about the book.



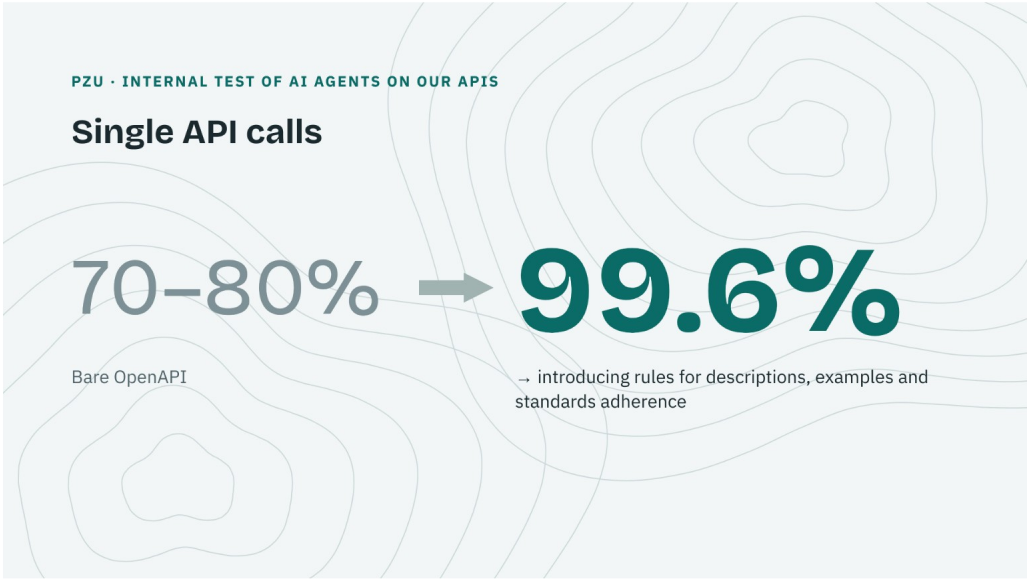
0:35 → 0:40. Five seconds: the book covers API design, lifecycle and governance, and this talk is one chapter of it in practice. The QR code goes to Amazon. Transition: and here's why governance matters more this year than ever.



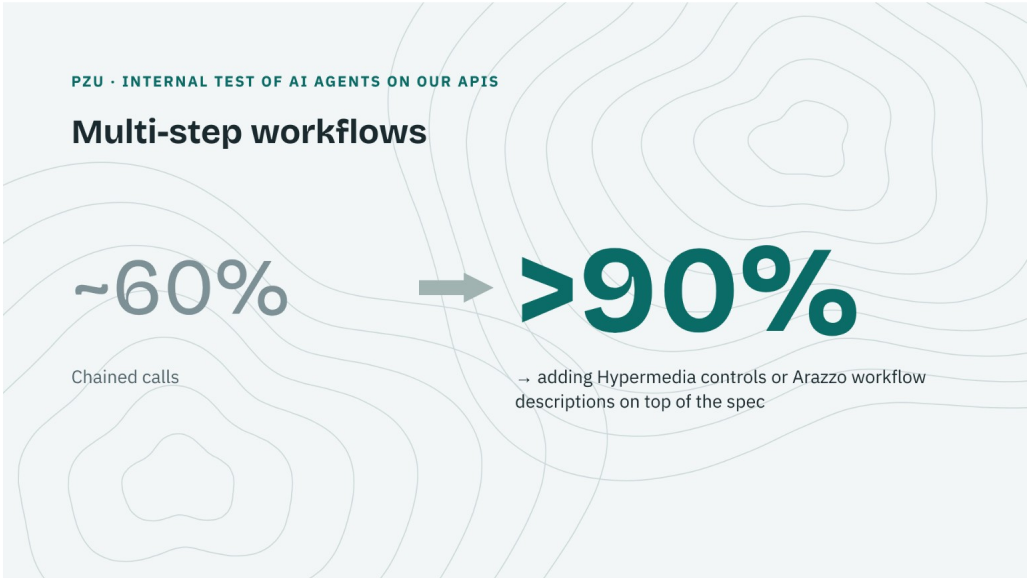
0:40 → 1:05. Agents are starting to call our APIs on their own, not just talk about them. A human developer reads the docs, pings someone on Slack, works around a gap. An agent can't. Whatever your contract says, or doesn't say, is what it does. So the quality of your API contracts is suddenly a business risk. Transition: so, a question from the bridge.



1:05 → 1:15. Bridge of Death. Read the question in the bridgekeeper's voice. Click: the comeback, like 'African or European?'. It sets up the next two slides: single calls, then workflows. Transition: good question, because we measured both.



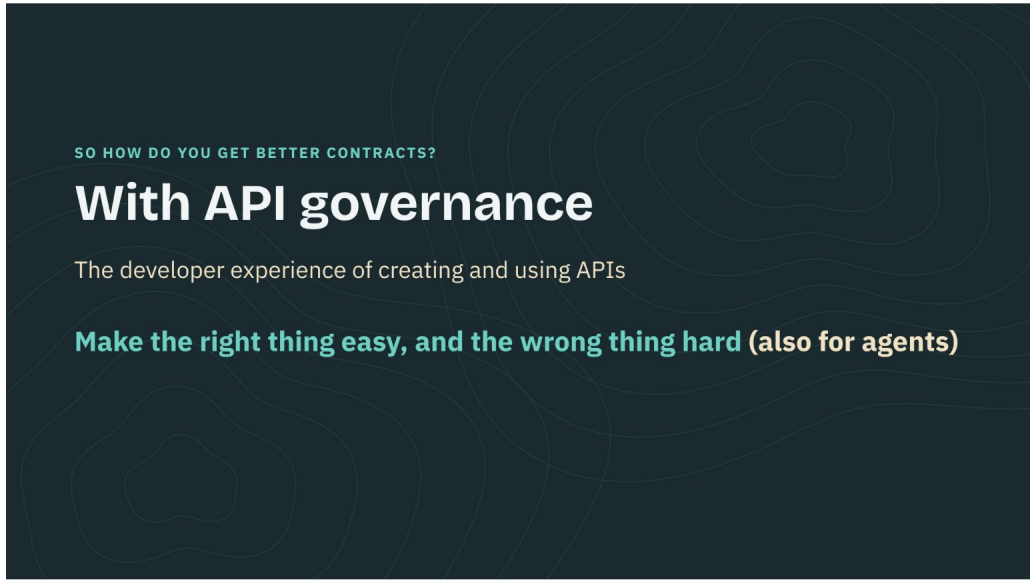
1:15 → 1:35. At PZU we pointed AI agents at our own OpenAPI files. Bare specs with minimal descriptions: 70 to 80 percent accuracy. Pause. Click: the same specs with what our governance program already required, descriptions everywhere, Problem Details for errors, examples: 99.6 percent. Transition: and it gets more interesting with workflows.



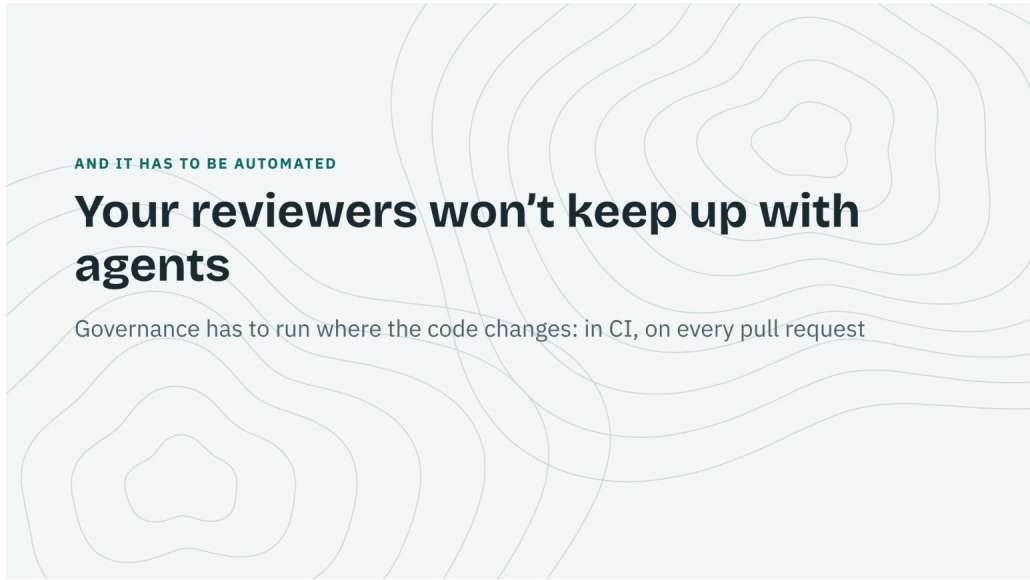
1:35 → 1:50. Multi-step workflows were worse, about 60 percent. Click: Arazzo workflow descriptions on top of the same specs took them above 90. We didn't start the program for AI; the accuracy came as a by-product. Transition: and here's why: an agent won't ask.



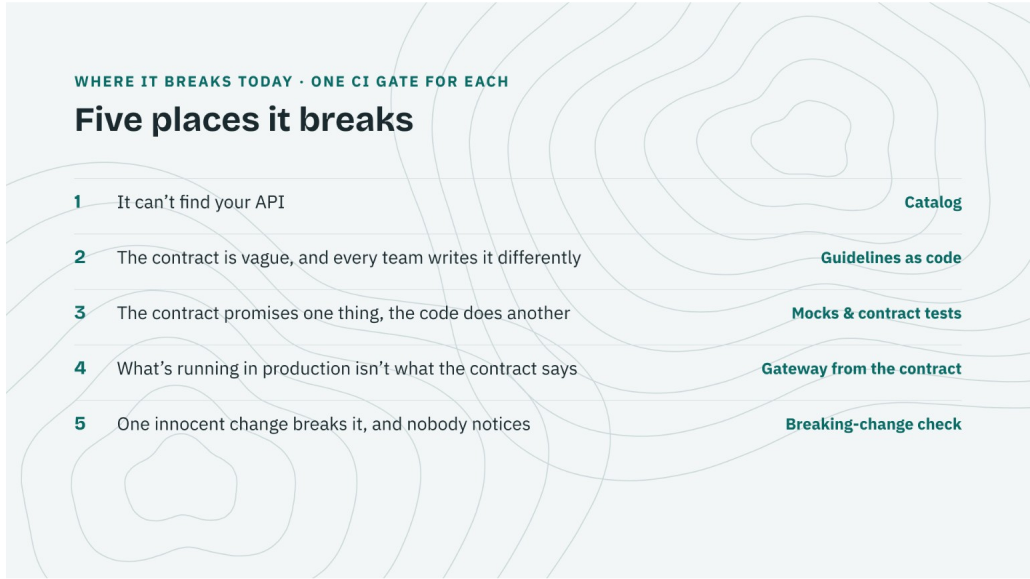
1:50 → 2:05. So here's the thing. Click: put those numbers together. Agents are fast becoming the biggest consumers of our APIs, and they only know what the contract tells them. So the quality of your API contracts stops being a nice-to-have. It is the product. Click: every vague description, missing example or undocumented error turns into a wrong call. Transition: so how do you get better contracts?



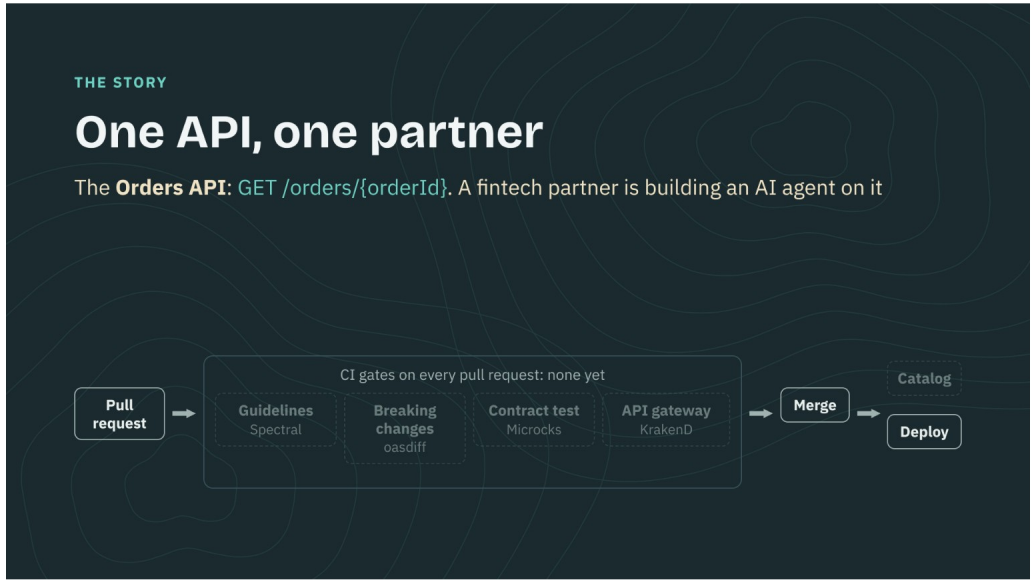
2:05 → 2:20. So how do you get better contracts? If the contract is the product, you need a way to make every contract good, not just the ones a few experts touched. Click: that's API governance. Click: to me it's the developer experience of creating and using APIs. Click: make the right thing easy, and the wrong thing hard, also for agents. Transition: and it has to be automated.



2:20 → 2:35. And it has to be automated. Click: agents call at machine speed, teams ship many times a day. A guideline document and reviewers who answer in two weeks won't keep up. Click: so governance has to run where the change happens: in the pipeline, on every pull request, in seconds. Transition: so which gates?



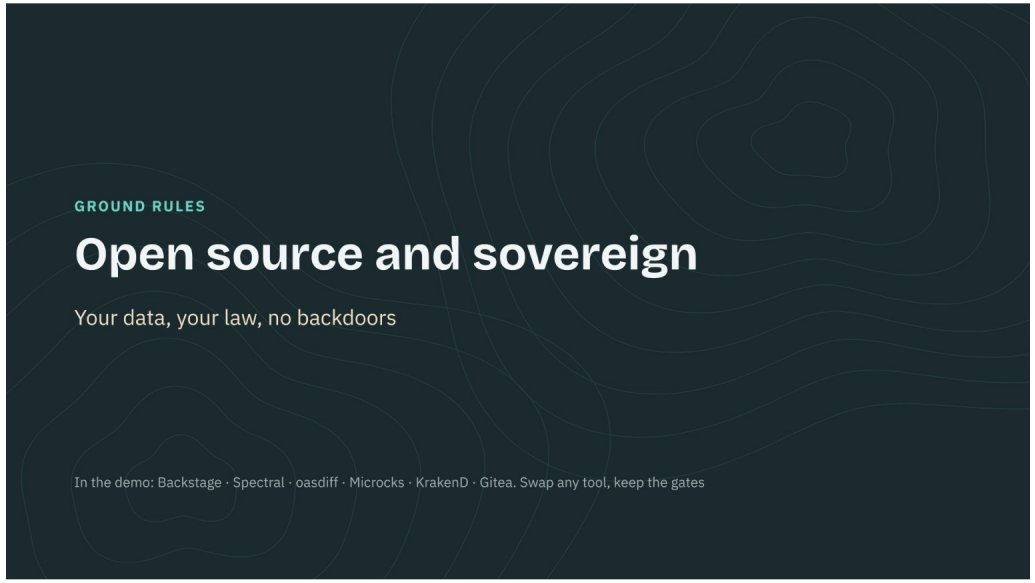
2:35 → 3:00. So which gates? Start with where agents break. Click: five places. Then click once per row; each break appears with the gate that fixes it. It can't find your API in the first place. The contract is vague, or every team writes it their own way. The contract promises a field the code never returns. What's deployed has quietly drifted from what's documented. Or someone ships a small, innocent change, and the agent falls over at 3 a.m. while nobody notices. Today we add one gate for each. Transition: let me introduce our API.



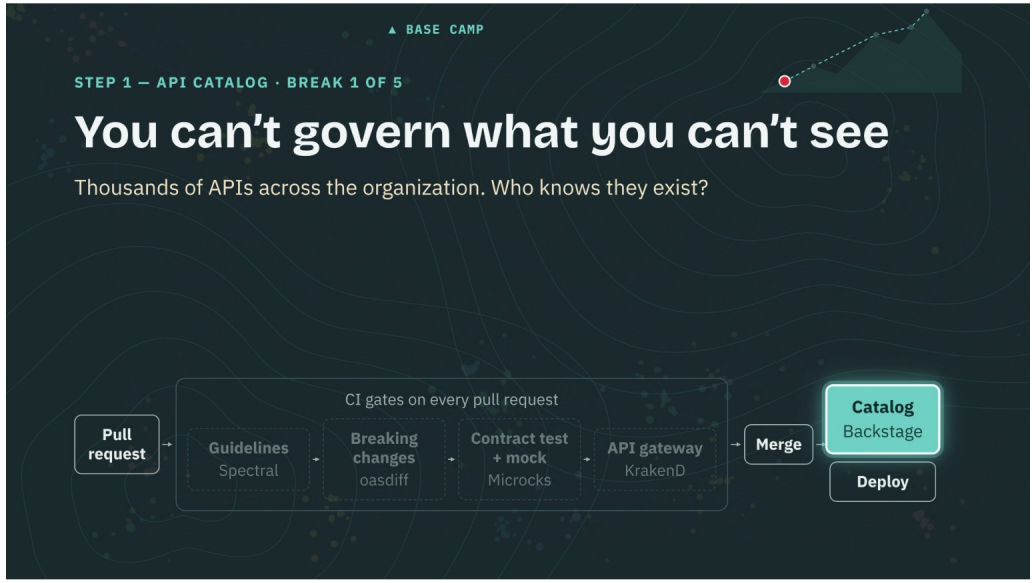
3:00 → 3:30. Meet the Orders API. One endpoint, GET an order by id. And a partner, a fintech building an AI agent on top of it. Their agent will hit every one of those five breaks. Click through the pipeline as you say it: pull request, CI, merge, deploy. Dashed boxes: no gates yet. The demos are recordings of a real local stack; the whole repo is open source. Think of the five steps as a climb: base camp, three camps, summit. Transition: first, a few ground rules.



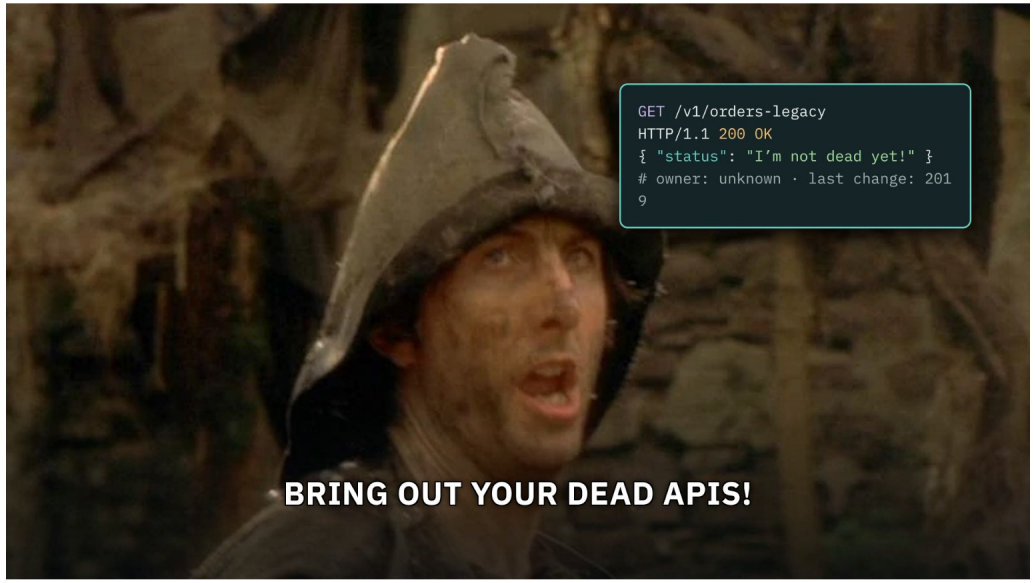
3:30 → 3:45. Click once per rule. Design-first: the contract comes first. OpenAPI is the single source of truth. Everything is configuration as code. And it's all automated in CI, on any CI system. Click: the small print. Transition: and one choice I care about.



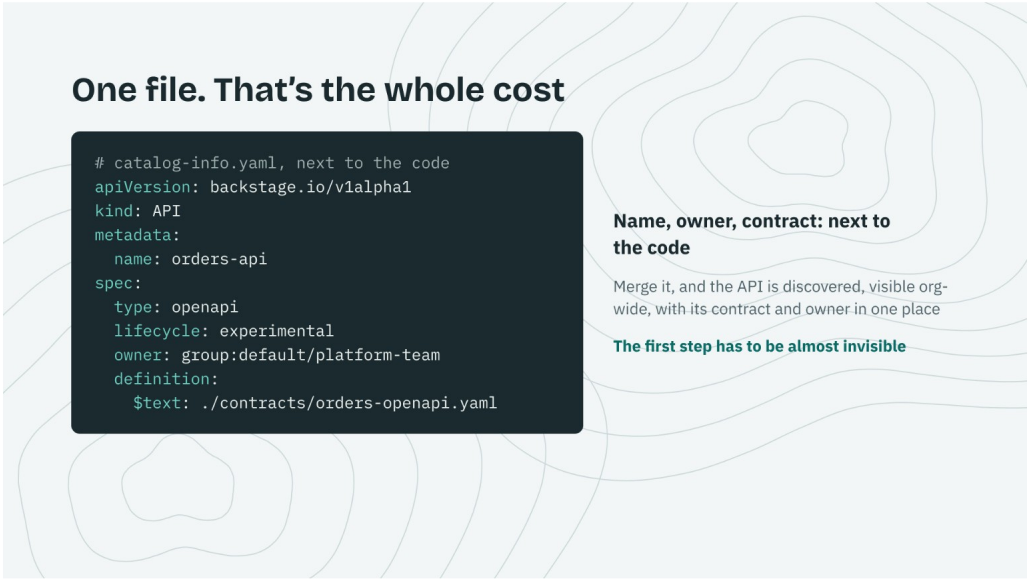
3:45 → 4:15. Click: open source and sovereign. Click: no lock-in, every tool in the demo can be replaced. Small tools that each do one job well, instead of months customizing one big platform. Your data stays where you choose, under the law you choose. And no backdoors, no telemetry, no one you didn't invite; you'll see what that means at step two. Even this repo lives on Codeberg, an EU forge. Transition: step one.



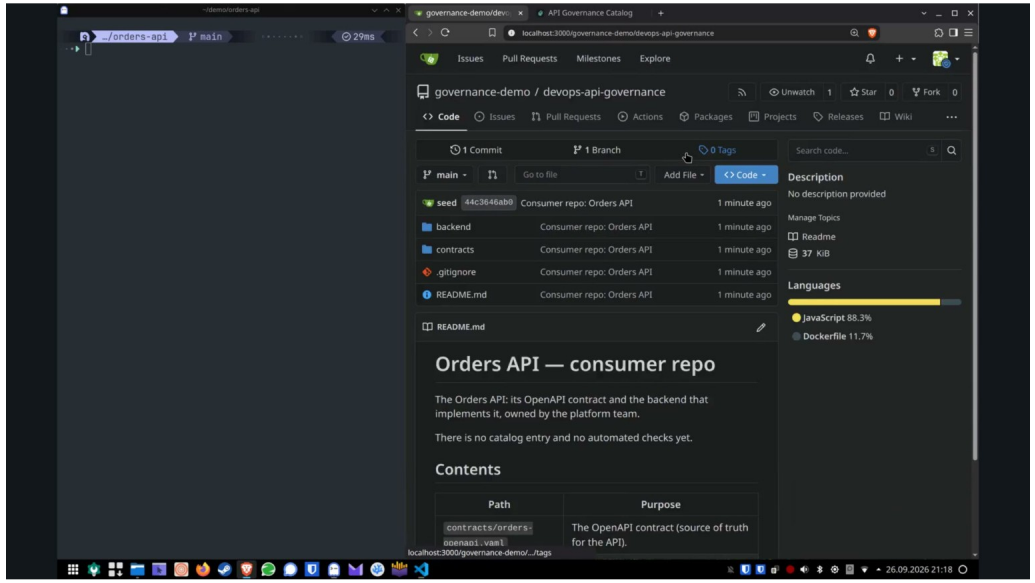
4:15 → 4:40. Behind me: an API dependency graph of an organization. Every dot an API. Nobody has the full map. Before any rule, you need to see what you have. Transition: bring out your dead.



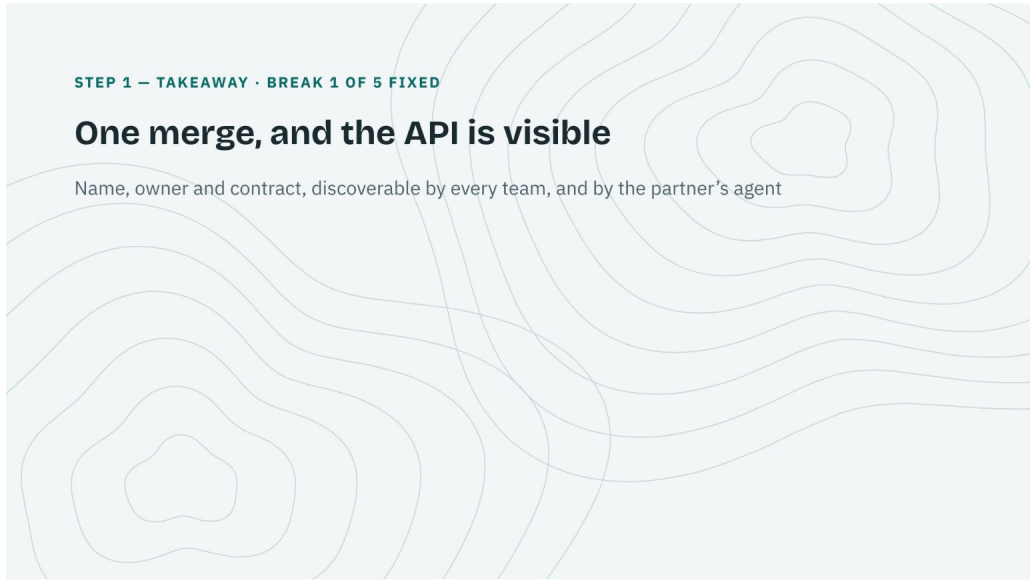
4:40 → 4:55. Bring out your dead! Every organisation has an API that should be gone. Click: and it answers. GET orders-legacy, 200 OK, I'm not dead yet. Nobody owns it, nobody remembers it, and an agent will happily call it. You can't retire what you can't see. Transition: and the cost of seeing it is one small file.



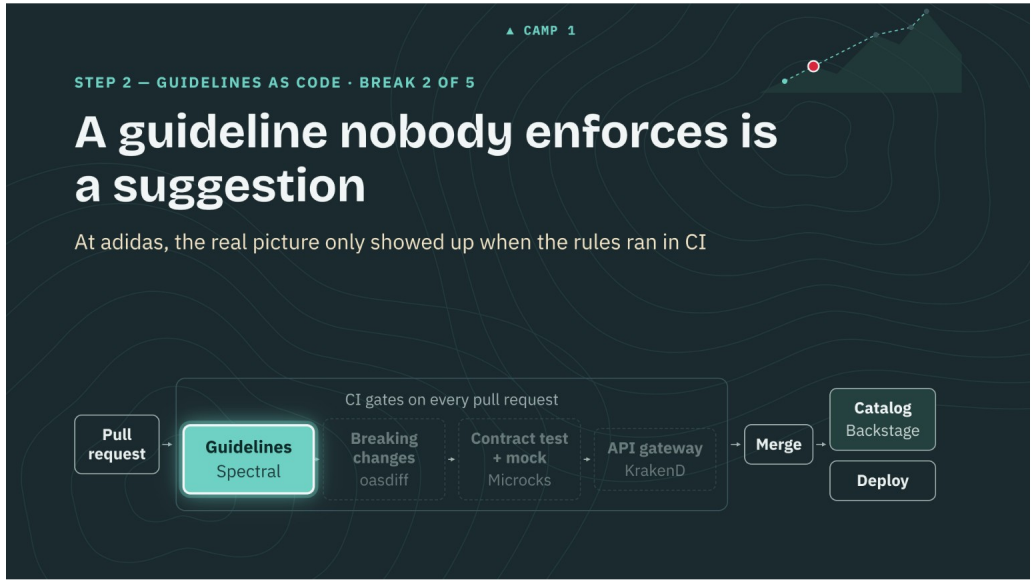
4:55 → 5:15. This is the entire cost of step one: one small file next to the code. Name, owner, and a pointer to the contract. Backstage here, but any catalog that reads files from repos works. Transition: let's watch it.



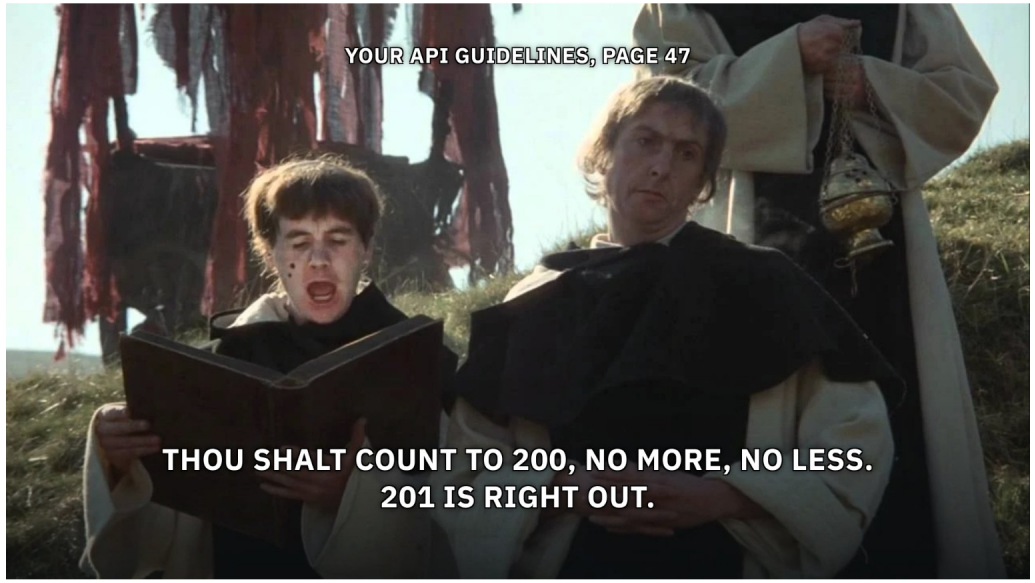
5:15 → 5:50. Click to play (35 s, sped up 1.25x). Cues: 0:00 the branch with catalog-info.yaml. ~0:10 open the file: that's all of it. ~0:17 PR, no checks yet, merge. ~0:25 Backstage APIs page, reload: orders-api appears by itself. ~0:30 the Definition tab: the contract, rendered. Say: commit, merge, done. Move on once the definition shows (the clip loops). Transition: the takeaway, in one line.



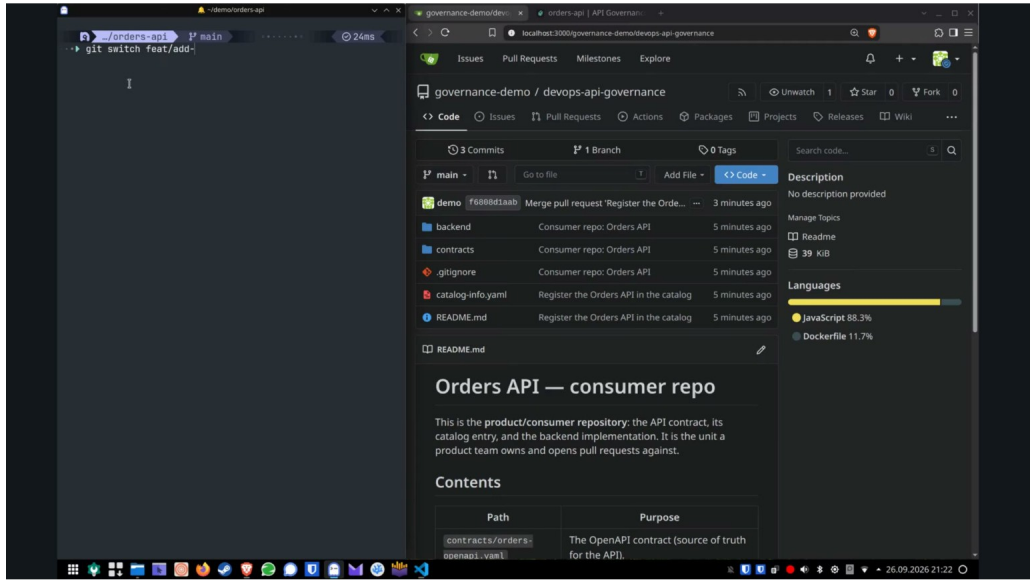
5:50 → 5:55. Five seconds: one file, one merge, and nobody has to ask where the API is. Transition: now the story from adidas.



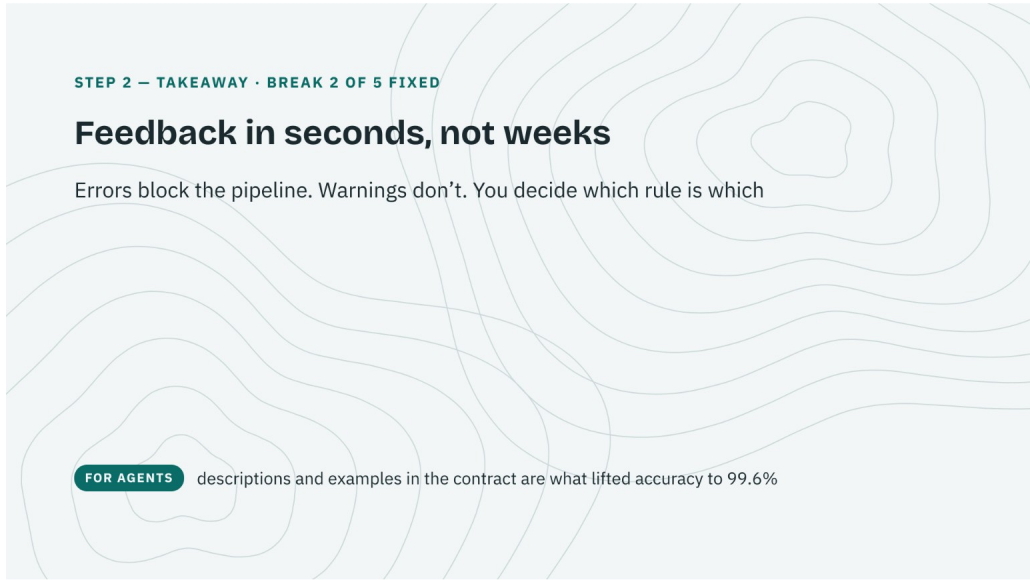
5:55 → 6:40. The adidas story, about 40 seconds. We had API guidelines. They existed, people knew them. But they were followed closely only when a team came to the API team for help with the design. Then we built an API contract repository that also checked how APIs followed the rules. For the first time we saw the real picture, and it was quite incomplete. That was step one: you can't govern what you can't see. The change came only when Spectral linting and contract testing moved from the guidelines document into the CI pipelines. Transition: or, as the Book of Armaments puts it.



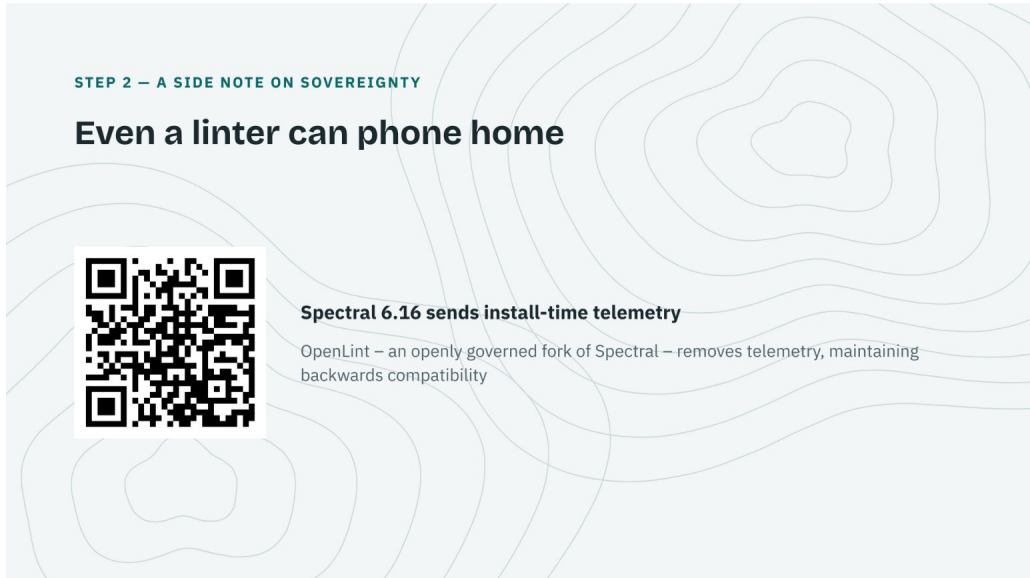
6:40 → 6:50. The Book of Armaments: the rules are clear, written down, even read out loud. Thou shalt count to 200, no more, no less; 201 is right out. And still, someone ships a 201 for a read. That's a guideline nobody enforces. Transition: so let's enforce it. Here's that gate.



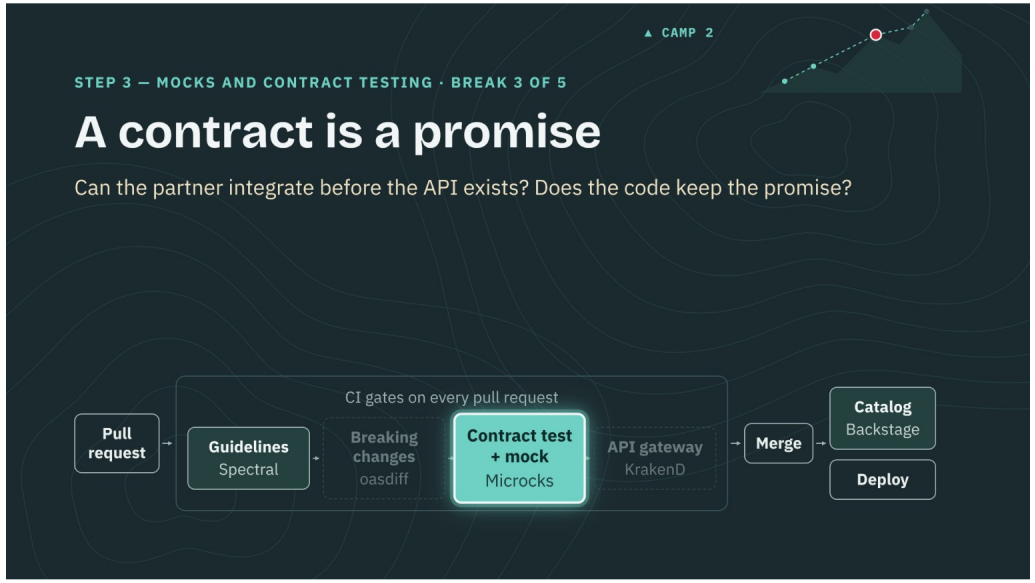
6:50 → 7:58. Click to play (68 s). Cues: 0:00 switch to the branch; git diff --stat shows the one new file, the CI workflow; push. ~0:08 (fast) open the PR that adds the Spectral gate. ~0:12 Backstage, API Guidelines: the rules live in the catalog, as docs. ~0:17 (fast) a change moves the server URL to http://. ~0:30 red check. ~0:32 the log: one finding, api-peak:rest17:2025-https-required, with a link. ~0:39 the link opens the same rule in Backstage: one source of truth for humans and CI. ~0:47 (fast) one-line fix, push, green, merge. Move on at the merge. Transition: so what does that give you?



7:58 → 8:23. The review board used to find this weeks later. Now it's a red check in seconds. Errors block the merge, warnings only inform, and you decide which rule is which: start with warnings, promote rules to errors once teams are clean. Click: for agents, the descriptions and examples these rules demand are what lifted accuracy to 99.6. Transition: one side note on sovereignty.



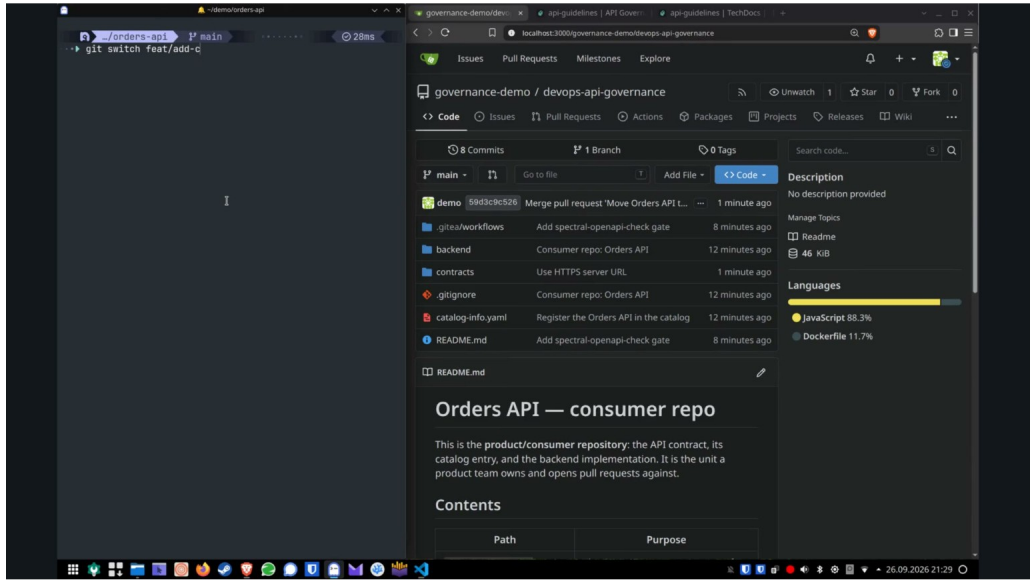
8:23 → 8:33. Ten seconds: the Spectral CLI we pin started sending install-time analytics through Scarf this summer. The CI image opts out; OpenLint is an openly governed fork with the telemetry removed. Transition: now, a contract is a promise.



8:33 → 8:58. Two questions for our fintech partner. One: can they start before the API is built? Two: once it's built, does the code do what the contract says? Microcks answers both from the same contract. Transition: and when the code doesn't keep the promise...



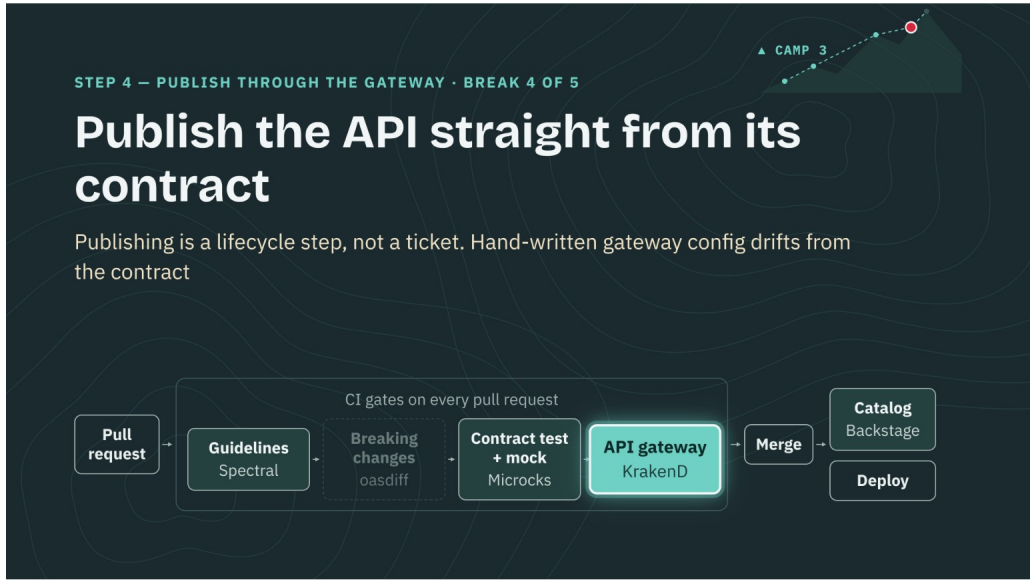
8:58 → 9:08. A contract is a promise. And when the code doesn't keep it, the developer says: it's not broken, it's resting, it's pining for the fjords. The contract test doesn't argue. Transition: let's see who's right.



9:08 → 10:48. Click to play (100 s). Cues: 0:00 switch to the branch; git diff --stat: the workflow grows by the contract-test job; push. ~0:10 the PR adds the contract-test gate. ~0:14 curl the mock: a live response from the contract's example; the partner integrates today. ~0:20 Microcks: the mocked service. ~0:30 (fast) a change promises a new field, currency. ~0:43 (fast) contract-test red. ~0:46 Microcks test detail: required property currency not found. ~1:06 curl the mock (has currency) vs the backend (doesn't): the code doesn't keep the promise. ~1:15 (fast) fix: don't promise currency until the backend returns it; green, merge. Move on at the merge. Transition: one example, many uses.



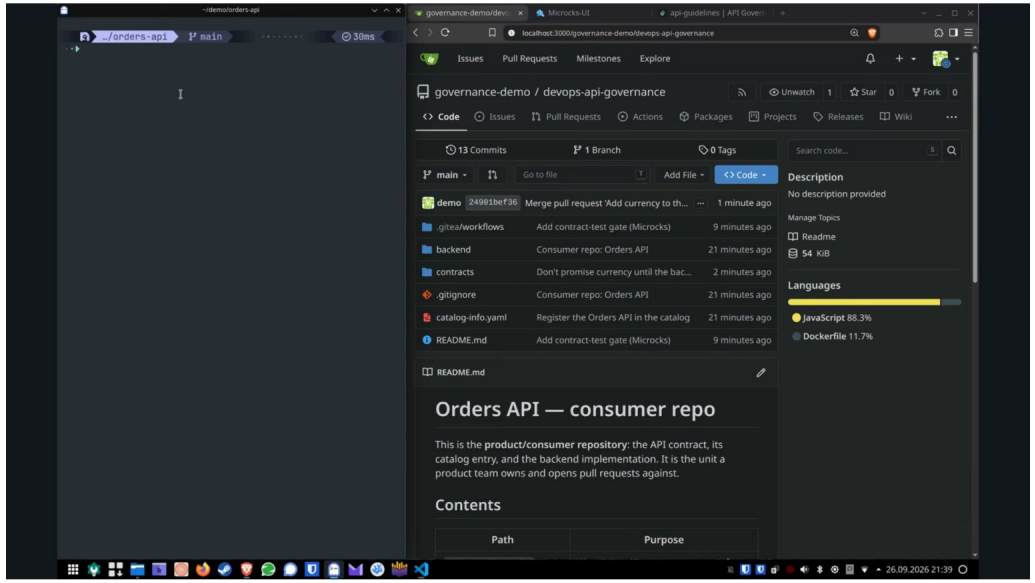
10:48 → 11:13. One example in the contract pays twice: it's the mock partners use, and it's the test the code must pass. Put that mock in your developer portal next to the docs, so partners can start on day one. Click: and the same goes for agents. Transition: now let's expose it for real.



11:13 → 11:38. Putting an API behind the gateway is how it gets published in its lifecycle. In most companies someone types routes into the gateway by hand, and nobody checks them against the contract. So we generate the gateway config from the contract, in CI, on every PR, and prove it. Transition: because the hand-written way looks like this.



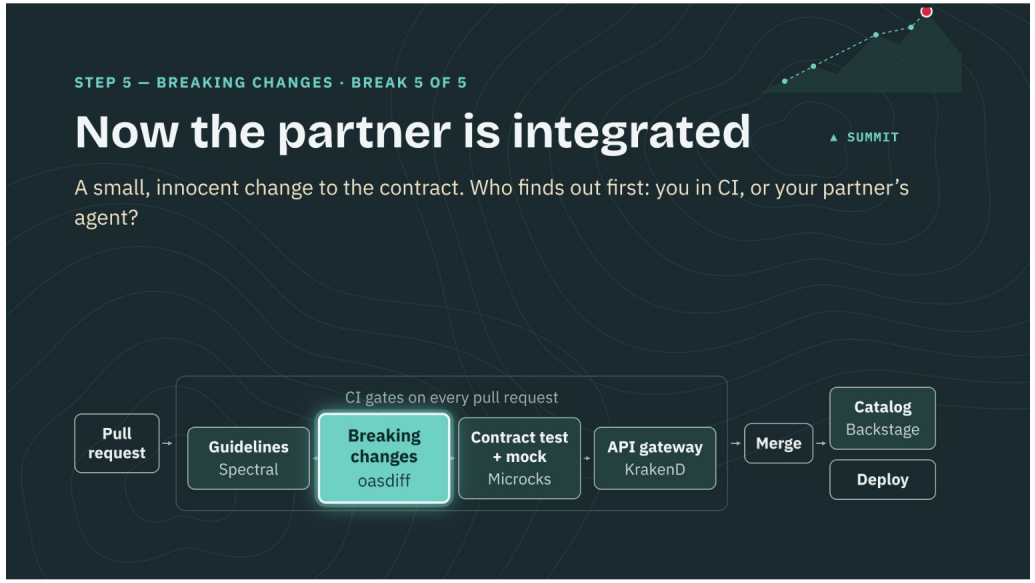
11:38 → 11:48. The Trojan Rabbit: they built it, wheeled it to the gate, and forgot to get inside. That's hand-written gateway config: deployed, impressive, and the API isn't in it, or not the one in the contract. Transition: so let's generate it. Watch.



11:48 → 12:37. Click to play (49 s). Cues: 0:00 curl through KrakenD: 404, no routes yet. ~0:04 (fast) the PR adds the gateway gate. ~0:20 (very fast) the job: generate krakend.json from the contract, validate it with krakend check, deploy it, then run the same contract test through the gateway. ~0:26 green, merge. ~0:31 curl again: 200 through KrakenD, same body as the backend. Move on after the second curl. Transition: generated, deployed, proven.



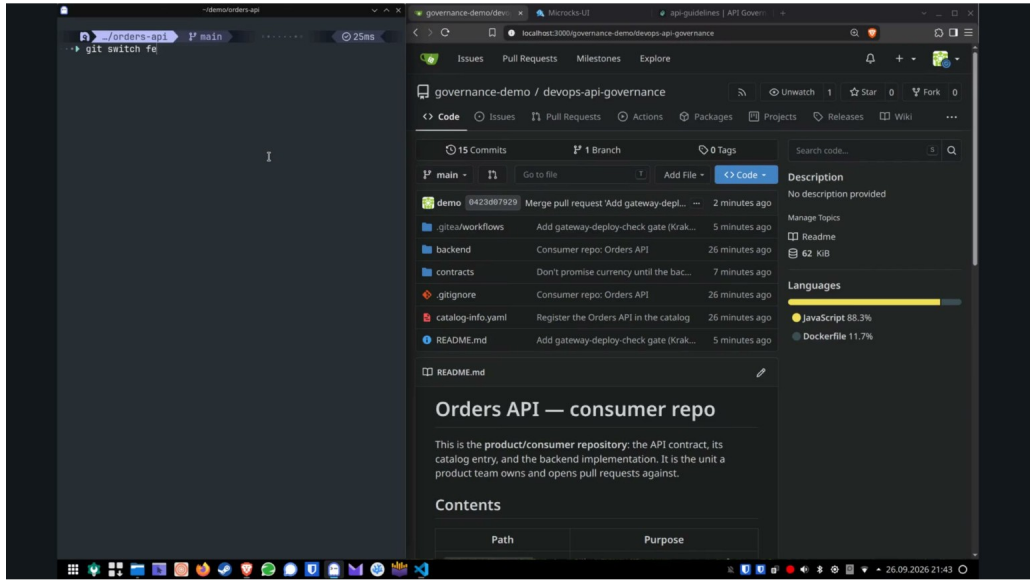
12:37 → 13:02. Catalog, tests, mock and now the gateway all come from the same contract. The contract defines what's on the gateway, and it can carry runtime configuration too. Another gateway, Kong or Apigee, just means another generator from the same OpenAPI. Click: for agents, production is exactly what they read. Transition: and now our partner is live.



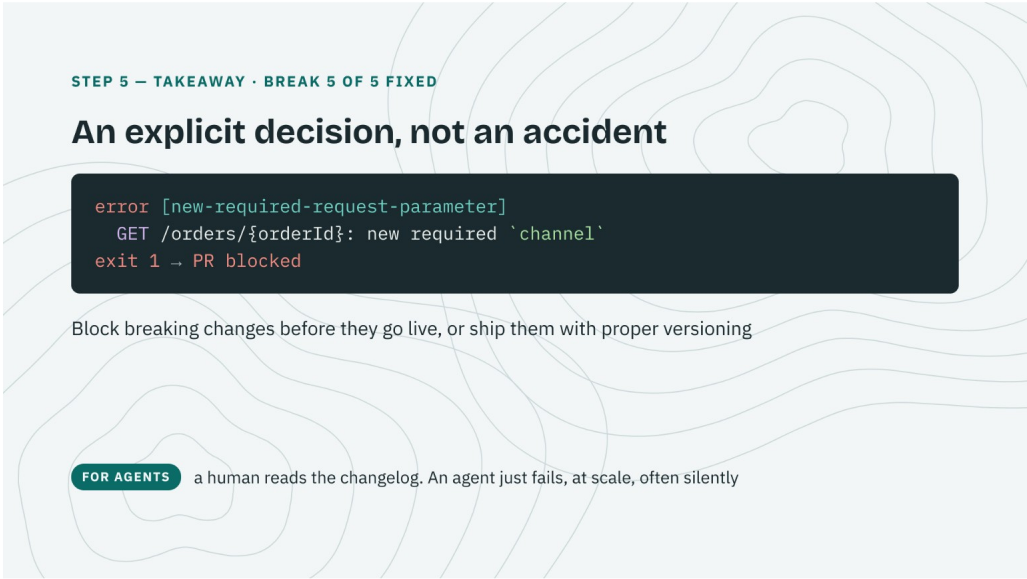
13:02 → 13:22. The fintech partner's agent is in production, calling us. Now someone makes a small change to the contract. Note where this gate sits: right after lint, before the runtime tests. A change that breaks clients shouldn't even reach them. Transition: and the author says…



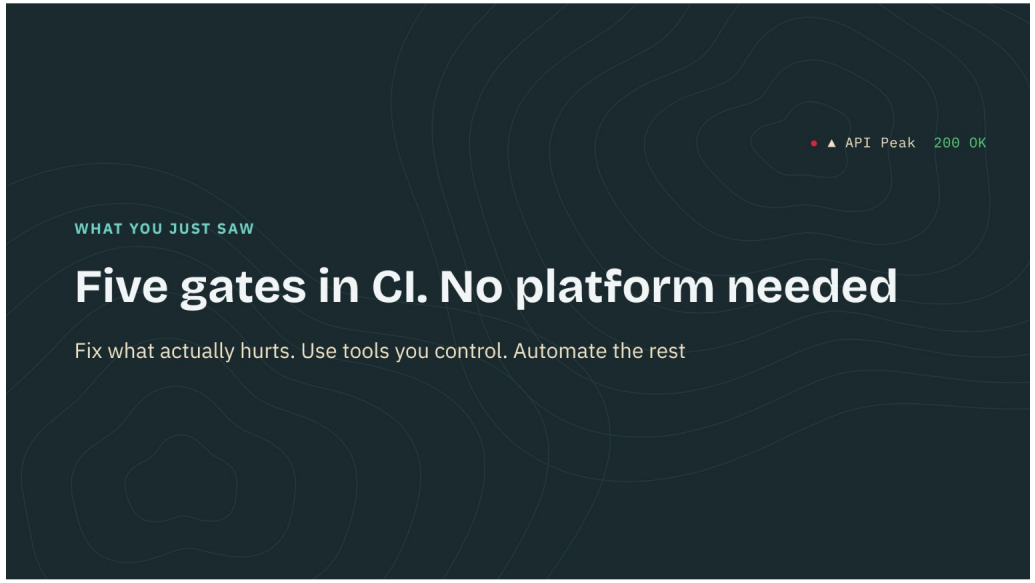
13:22 → 13:30. 'Tis but a scratch. Every breaking change starts like this: just a small change, one new parameter. Ask the partner's agent how small it is. Transition: watch.



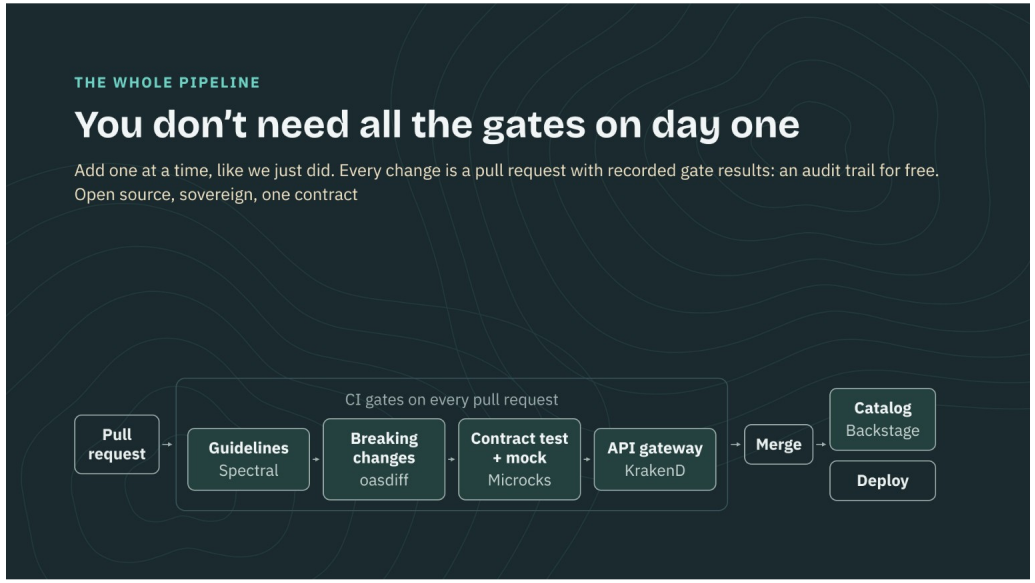
13:30 → 14:44. Click to play (74 s). Cues: 0:00 switch to the branch, git diff --stat, push. ~0:09 (fast) open the PR. ~0:13 Files changed: the new breaking-changes job in the middle of the chain, needs repointed. ~0:25 (very fast) four gates green, merge. ~0:30 the innocent change: a new query parameter channel, required. ~0:44 breaking-changes red; contract test and gateway Skipped, grey: the later gates don't even run. ~0:46 (2x) oasdiff: new-required-request-parameter, channel. ~0:54 (2x) fix: make channel optional, push. ~1:05 (very fast) all four green, merge. Move on at the merge. Transition: blocked before anyone got hurt.



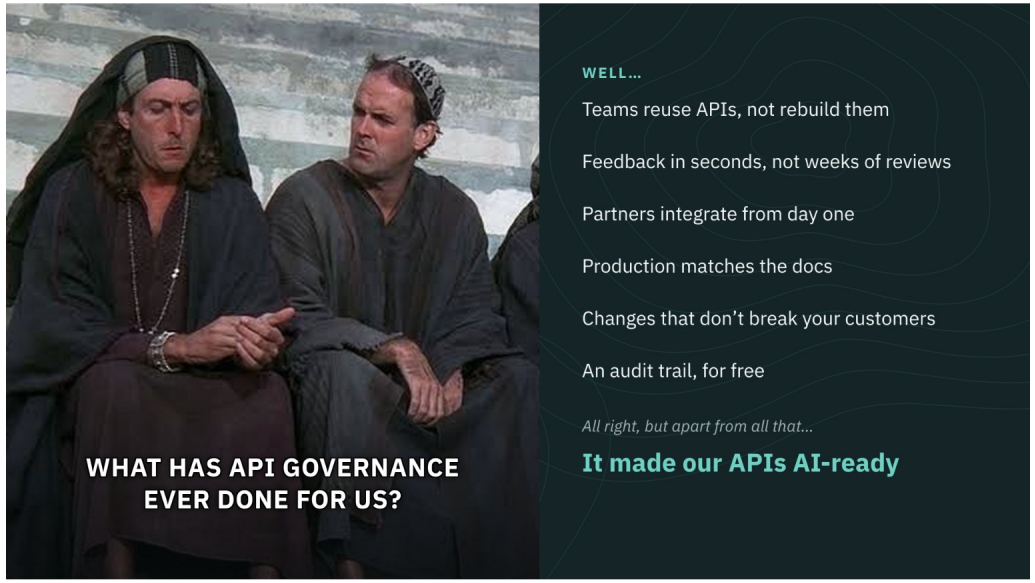
14:44 → 15:09. Clients that don't send channel today would start getting 400s. CI caught it before the partner did. If you truly need the change, make it a decision: a new version, run both, deprecate. Click: a human reads the changelog; an agent just fails. Transition: let's step back.



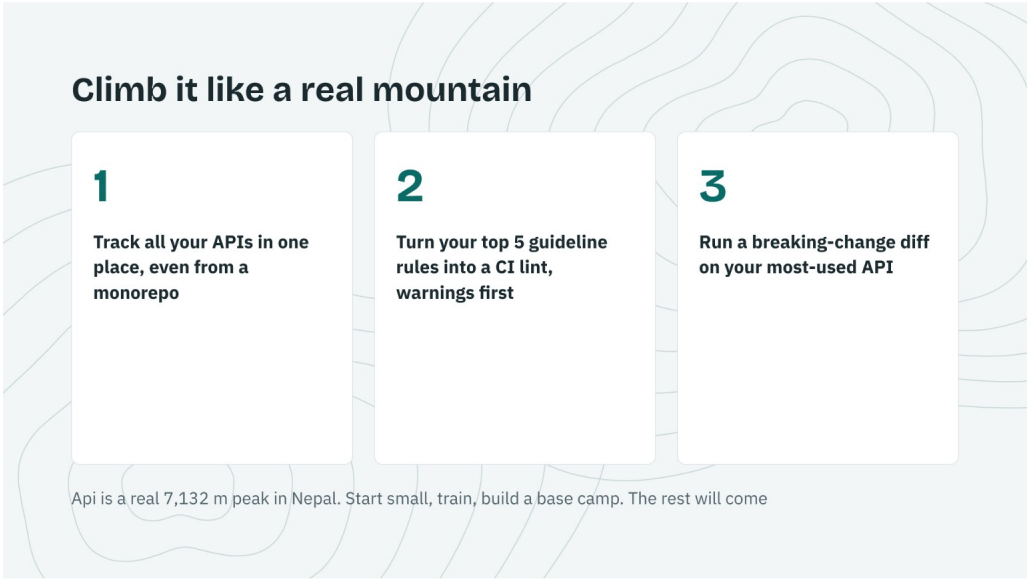
15:09 → 15:24. That's the whole thing: five gates in CI, no huge platform that tries to solve every possible problem. Address what actually hurts you, pick tools you can control, and let CI do the enforcing. Transition: here's the whole pipeline.



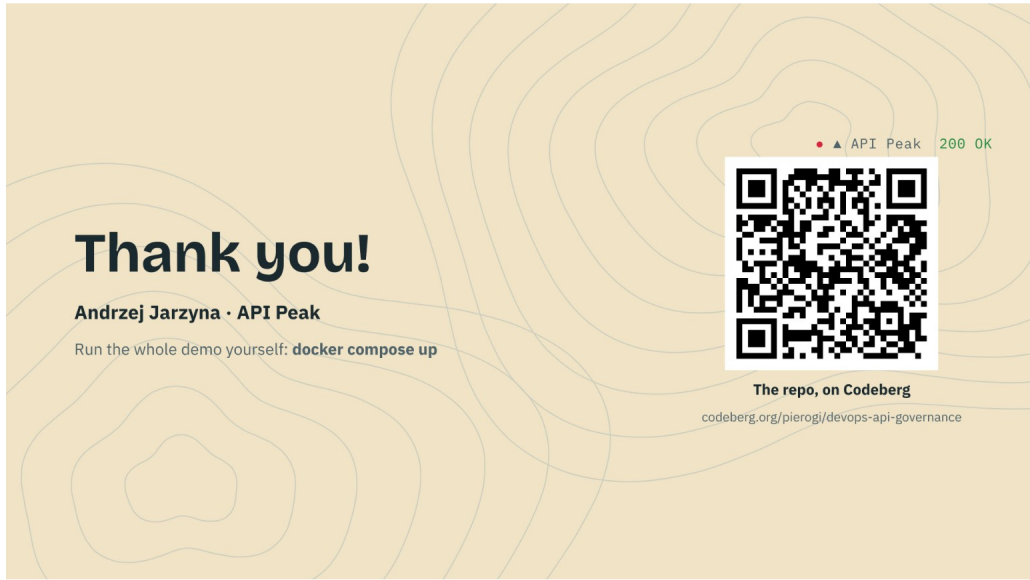
15:24 → 15:49. Five gates, one contract. And the order we added them is the rollout plan: start minimal, then nudge. For regulated industries, every contract change is a PR with recorded gate results, an audit trail you get for free. Transition: so, what has it ever done for us?



15:49 → 16:19. Reg's question: what has API governance ever done for us? Click once per answer and let it build, like the meeting in the film: teams reuse APIs instead of rebuilding them, feedback in seconds, partners on day one, production matches the docs, no broken customers, an audit trail for free. Click: all right, but apart from all that... Click: it made our APIs AI-ready. Transition: so what can you do on Monday?



16:19 → 16:59. API Peak is named after Api, a real 7,132-metre mountain in far-west Nepal, and building a governance pipeline is the same kind of climb. Nobody starts at the summit: start small, train, build a base camp, and the rest will come. Your base camp for Monday, three things, no big program needed. One: track all your APIs in one place, even from a monorepo. Two: take your top five guideline rules and make them a CI lint, as warnings first. Three: run a breaking-change diff on your most-used API and see what you've been shipping. Transition: thank you.



16:59 → Q&A. Thank you. The QR code is the repo: run the whole demo with docker compose. Leave this slide up during questions.